

Q1: Heuristic design and admissibility

Notation: Let n be the current word, g the goal word, $D(n)$ the set of positions where n and g differ, $d = |D(n)|$, and $h^*(n)$ the true optimal remaining cost from n .

Steps

Formula: $h(n) = d$, the number of positions where n differs from g (the Hamming distance).

Lower bound: the number of actions still needed to reach g .

Why it can't overestimate: An action changes only one letter, so it can reduce d by at most one. It may not reduce d at all, if it changes a correct letter or writes a letter that still isn't the goal's. Any path to the goal therefore needs at least d actions, and each costs exactly 1, so $h^*(n) \geq d$. The dictionary sometimes forces a detour; for example, why $\rightarrow$ ask takes 5 steps although $d = 3$. A detour only adds actions, so the bound still holds. At the goal $d = 0$, so $h(g) = 0$.

Scrabble

Formula: $h(n) = \sum_{i \in D(n)} \text{points}(g[i])$, the sum of the Scrabble values of the goal letters at the mismatched positions.

Lower bound: the cost of the final insertion of the correct goal letter at each mismatched position.

Why it can't overestimate: For every mismatched position i , some action on the path has to place $g[i]$ at position i , since otherwise the word never matches the goal there. Scrabble cost charges for the letter being inserted, so that action costs exactly $\text{points}(g[i])$. An action touches one position, so each mismatched position needs its own action, and these costs add up. Any extra moves (e.g., detours, passing through other letters, or undoing a correct letter and redoing it) cost at least 1 point each and only increase the total. The true remaining cost is therefore at least $h(n)$. At the goal there are no mismatched positions, the sum is empty, and $h(g) = 0$.

Frequency

Formula: Let $r_{\min} = \min_w \text{rarity}(w)$ over all words in the dictionary (≈ 0.0162 , for "the"). Then:

$$h(n) = 0 \text{ if } d = 0$$

$$h(n) = (1 + \text{rarity}(g)) + (d - 1)(1 + r_{\min}) \text{ if } d \geq 1$$

Lower bound: the exact cost of the final step into g , plus the minimum possible cost of each other required step.

Why it can't overestimate: Every solution path ends by entering g , and that step always costs exactly $1 + \text{rarity}(g)$. If $d \geq 1$, the steps argument shows at least $d - 1$ other actions are needed. Each of those enters some word w at cost $1 + \text{rarity}(w) \geq 1 + r_{\min}$, since $r_{\min}$ is the smallest rarity in the dictionary. A solution with $k \geq d$ moves therefore costs at least $(1 + \text{rarity}(g)) + (d - 1)(1 + r_{\min}) = h(n)$. The goal's rarity is used only for the final step; charging it for every step would overestimate whenever the intermediate words are more common than the goal. At the goal, $h = 0$ by definition.

Q2: Does the heuristic actually help?

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
why, ask	Steps	5.00	773	5.00	16	97.9%
why, ask	Scrabble	9.00	765	9.00	13	98.3%
why, ask	Frequency	38.86	808	38.86	634	21.5%
bear, duck	Steps	4.00	691	4.00	6	99.1%
bear, duck	Scrabble	11.00	2981	11.00	6	99.8%
bear, duck	Frequency	38.24	1084	38.24	301	72.2%

UCS and A* same optimal cost? Yes, in all six rows. Logic follows from the Q1 consistency argument, the supplied search never re-expands a state once explored, so it guarantees optimal answers only when the heuristic is consistent.

Largest reduction: Bear, duck (Scrabble) was the highest reduction overall. Further, all instances of word pairs with the Scrabble cost measure represented the highest reduction (98.3% on why, ask. 99.8% on bear, duck).

Least informative cost measure: Frequency for both pairs, with the lowest reduction at 21.5% for why, ask, and 72.2% for bear, duck.

Explanation of differences:

- Steps and scrabble count letters that still need to change, and in the context of these problems, nearly every step fixes one of them:
 - On bear, duck there are no detours, meaning both heuristics equal the true cost
 - On why, ask the dictionary forces a 2-step detour. Both heuristics miss those extra 2 steps, but they're close
 - Scrabble beats steps slightly on why, ask because it weighs each missing letter by its actual value
 - Bear, duck (scrabble) shows the biggest reduction because UCS performs the worst there. Scrabble costs vary from (1, 10), so there are many "cheap" (looking) paths to explore
- Frequency charges the goal's rarity only for the last step. Every other step is priced at $1 + r_{\min} \approx 1.016$ (the cost of entering "the"). Actual intermediate words are much rarer, so the true cost is several times larger. A* ranks nodes by $g + h$, and when h is small next to g , the ordering is close to UCS's.

- why, ask gets the smaller benefit because the optimal frequency path is long, and each step through rare words adds cost the heuristic can't see

Q3: Reversibility and direction-dependent costs

Steps: Yes. The justification comes from the original heuristic definition. If you reverse any path from W1 to W2, you get a legal path from W2 to W1, since actions are reversible. Also, since it has the same number of moves (and each move costs 1), the cost is identical. This logic applies to the optimal path as well, thus the two optimal costs are equal.

Scrabble: No. Counterexample:

```
dc(cog, dog, scrabble) cost:2.00; time:0.000; solution:cog dog; deltas:[0.0]
```

```
dc(dog, cog, scrabble) cost:3.00; time:0.000; solution:dog cog; deltas:[0.0]
```

These are the optimal costs, since A* is optimal with the heuristic, not just the cost of one particular path.

Frequency: No. Counterexample:

```
dc(cog, dog, frequency) cost:6.82; time:0.000; solution:cog dog; deltas:[0.0]
```

```
dc(dog, cog, frequency) cost:12.39; time:0.000; solution:dog cog; deltas:[0.0]
```

Both of these runs are one-step solutions, but the cost is different. That is because the cost of a transformation using frequency is based on the rarity of the word you arrive at. Cog is a rarer word than dog, thus moving towards it costs more.

For each cost measure, all actions are reversible. The difference lies in the cost symmetry, which changes for each cost measure.

Q4: LLM heuristic audit

1. Model, prompt, proposed heuristic

Model: Gemini Flash 3.8

Prompt: Exact prompt given in the assignment PDF. (copy+pasted)

Proposed heuristic: h_0 with two rarity terms: the rarity of the goal word, which is always paid on the final step, and the global minimum rarity $r_{\min}$ charged once for each of the $h_0(n) - 1$ intermediate words the path must pass through. The model also noted an optional variant replacing $(h_0(n) - 1) * r_{\min}$ with the sum of the $h_0(n) - 1$ smallest rarities in $D \setminus \{n, g\}$ and a fallback term $r_{\text{pred}}(g)$ for the case $\text{rarity}(g) = 0$ and $r_{\min} = 0$

2. Formula

Let n be the current word, g the goal, and $d = h_0(n)$ (the number of positions where n and g differ), and $r_{\min} = \min$ over all dictionary words w of $\text{rarity}(w)$

$$h_1(n) = 0, \text{ if } n = g$$

$$h_1(n) = d + \text{rarity}(g) + (d - 1) * r_{\min}, \text{ if } n \neq g$$

The model wrote the last term as $\max(0, d - 1) * r_{\min}$; since $d \geq 1$ whenever $n \neq g$, the max has no effect

3. Admissibility

h_1 is admissible.

Proof: If $n = g$, $h_1(g) = 0 = h^*(g)$. If $n \neq g$, consider any path $n = w_0, w_1, \dots, w_k = g$. Each action changes exactly one letter, so it can fix at most one of the d mismatched positions; therefore $k \geq d$. The path's cost is:

$$\sum_{i=1..k} (1 + \text{rarity}(w_i)) = k + \text{rarity}(g) + \sum_{i=1..k-1} \text{rarity}(w_i),$$

since the last word entered is g . Every rarity is at least $r_{\min}$, and there are $k - 1 \geq d - 1$ intermediate words, so:

$$\text{cost} \geq d + \text{rarity}(g) + (d - 1) \cdot r_{\min} = h_1(n).$$

This holds for every path from n to g , including the optimal one, so $h_1(n) \leq h^*(n)$.

h_1 is also more informative than h_0 : for $n \neq g$, $h_1(n) - h_0(n) = \text{rarity}(g) + (d - 1) * r_{\min} \geq \text{rarity}(g) > 0$, because every word in the dictionary has a positive rarity. So h_1 is strictly larger than h_0 at every non-goal state.

4. Repair

No repair needed.

5. Experimental comparison with h0 (frequency cost, A*)

Word pair	Heuristic	Solution cost	Nodes expanded	Reduction vs h0
why, ask	h0	38.86	755	-
why, ask	h1	38.86	634	16.0%
bear, duck	h0	38.24	720	-
bear, duck	h1	38.24	301	58.2%

Both heuristics returned the same optimal path cost and the same path on both problems. h1 expanded 16.0% fewer nodes on (why, ask) and 58.2% fewer nodes on (bear, duck). The gain is larger on (bear, duck) because its optimal path is short, so the known final step ($1 + \text{rarity}(\text{duck})$) makes up a large share of the true cost; for (why, ask), the 7-step optimal path passes through several intermediate words, whose rarities far exceed $r_{\min}$.

Evidently, h1 is provably admissible and consistent. Further, it never returned a worse solution than h0, and it reduced node expansions on both problems.

Generative AI Use Disclosure

System: Claude (Opus 5.5), in addition to Gemini Flash 3.8 (required for Q4)

Used for:

- Orientation and task planning
- Implementation guidance and code review (dc.py)
- Running correctness checks
- Edit & review of written answers for Q1-4
- Verifying Gemini heuristic against our heuristic for Q4

Verification: I examined the console output after each code change to confirm there were no errors or bugs, and compared solution costs against the sample outputs in the assignment handout.